Experiment No. 9

Software Engineering Tools and Technologies – Compare and Contrast Different Software Engineering Tools and Technologies

Aim

To study different software engineering tools and technologies, implement an Email and Password Login Flow using Python and using React.js + Node.js with static (fixed) credentials, and compare them briefly.

Objectives

- Understand the role of tools in the Software Development Life Cycle (SDLC).
- Identify categories of software engineering tools.
- Implement the same login requirement using two different technologies.
- Compare Python (console) with React.js + Node.js (web).
- Select suitable tools and technologies for a given scenario.

Theory

Software engineering tools are programs that help developers and teams to plan, design, write, test, deploy and maintain software efficiently. They reduce manual effort, improve quality and support teamwork.

Category	Purpose	Examples
Project Management	Plan tasks, track progress	Jira, Trello
Design / Modeling	UML diagrams, wireframes	draw.io, Figma
IDE / Editor	Write, debug and run code	VS Code, IntelliJ, Eclipse
Version Control	Track changes, teamwork	Git, GitHub, SVN
Build / Package Manager	Manage dependencies	npm, pip, Maven
Testing	Unit, API, UI testing	PyTest, JUnit, Postman, Selenium
CI/CD	Automate build and deployment	Jenkins, GitHub Actions

Python: a high-level, interpreted language with simple syntax. Used here for a console login.

Node.js: a JavaScript runtime. With Express it is used to build the backend API.

React.js: a JavaScript library for building user interfaces. Used for the login form.

Software and Hardware Requirements

Software: Python 3.10 or later, Node.js 18 or later (with npm), VS Code, Postman, web browser, Windows/Linux/macOS.

Hardware: Intel i3 or above, 4 GB RAM (minimum), 5 GB free disk space.

Problem Statement

Develop an Email and Password Login Flow. No database is used. A static (fixed) email and password are stored in the program:

Static Email	Static Password
test@gmail.com	123456

If the entered email and password both match, show "Login Successful". Otherwise show "Login Failed". Implement it in (A) Python and (B) React.js + Node.js.

Algorithm

- **Step 1:** Start.
- **Step 2:** Store static email = test@gmail.com and static password = 123456.
- **Step 3:** Take email and password as input.
- **Step 4:** Compare entered email AND password with the static values.
- **Step 5:** If both match, display "Login Successful".
- **Step 6:** Otherwise, display "Login Failed: Invalid email or password".
- **Step 7:** Stop.

Program A: Login using Python

File name: **login.py**. No external library is required. The user gets 3 attempts.

```
# login.py - Email and Password login using static credentials

STATIC_EMAIL = "test@gmail.com"
STATIC_PASSWORD = "123456"
MAX_ATTEMPTS = 3

def login(email, password):
    if email == STATIC_EMAIL and password == STATIC_PASSWORD:
        return True
    return False

print("========== LOGIN ==========")
attempts = 0
while attempts < MAX_ATTEMPTS:
    email = input("Enter Email    : ").strip()
    password = input("Enter Password : ").strip()

    if login(email, password):
        print("Login Successful")
        break
    else:
        attempts += 1
        print("Login Failed: Invalid email or password")
        print("Attempts left:", MAX_ATTEMPTS - attempts)
else:
    print("Too many failed attempts. Access blocked.")
```

Run: `python login.py` (or `python3 login.py`)

Sample Output 1 (correct credentials)

```
========== LOGIN ==========
Enter Email    : test@gmail.com
Enter Password : 123456
Login Successful
```

Sample Output 2 (wrong credentials)

```
========== LOGIN ==========
Enter Email    : test@gmail.com
Enter Password : 111111
Login Failed: Invalid email or password
Attempts left: 2
```

Program B: Login using React.js + Node.js

The backend (Node.js + Express) holds the static email and password and checks them. The frontend (React.js) shows the login form and sends the data to the backend using a POST request.

Project structure

```
login-app/
 |-- backend/
 |     |-- server.js
 |-- frontend/
       |-- src/
             |-- App.jsx
```

Step 1: Create backend

```
mkdir login-app
cd login-app
mkdir backend
cd backend
npm init -y
npm install express cors
```

File: backend/server.js

```
const express = require("express");
const cors = require("cors");

const app = express();
app.use(cors());           // allow requests from React (different port)
app.use(express.json());   // read JSON body

const STATIC_EMAIL = "test@gmail.com";
const STATIC_PASSWORD = "123456";

app.post("/api/login", (req, res) => {
  const { email, password } = req.body;

  if (email === STATIC_EMAIL && password === STATIC_PASSWORD) {
    return res.status(200).json({
      success: true,
      message: "Login Successful",
    });
  }
  return res.status(401).json({
    success: false,
    message: "Login Failed: Invalid email or password",
  });
});

app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

Start backend: `node server.js` (keep this terminal open).

Step 2: Create frontend (open a NEW terminal)

```
cd login-app
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev
```

File: frontend/src/App.jsx (delete old content and paste this)

```
import { useState } from "react";

export default function App() {
  const [email, setEmail] = useState("");
```

```
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");
  const [loggedIn, setLoggedIn] = useState(false);

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const res = await fetch("http://localhost:5000/api/login", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ email, password }),
      });
      const data = await res.json();
      setMessage(data.message);
      setLoggedIn(data.success);
    } catch (err) {
      setMessage("Server not reachable. Start the backend first.");
    }
  };

  if (loggedIn) {
    return (
      <div style={{ textAlign: "center", marginTop: 80 }}>
        <h2>Welcome, {email}</h2>
        <p>{message}</p>
        <button onClick={() => { setLoggedIn(false); setPassword(""); }}>
          Logout
        </button>
      </div>
    );
  }

  return (
    <form onSubmit={handleLogin}
          style={{ maxWidth: 320, margin: "80px auto" }}>
      <h2>Login</h2>
      <input type="email" placeholder="Email" value={email}
             onChange={(e) => setEmail(e.target.value)} required />
      <br /><br />
      <input type="password" placeholder="Password" value={password}
             onChange={(e) => setPassword(e.target.value)} required />
      <br /><br />
      <button type="submit">Login</button>
      <p>{message}</p>
    </form>
  );
}
```

Open `http://localhost:5173` in the browser. Backend (port 5000) and frontend (port 5173) must run at the same time.

Observation Table

Test Case	Email	Password	Expected	Python	React + Node
TC01	test@gmail.com	123456	Login Successful	Pass	Pass
TC02	test@gmail.com	654321	Login Failed	Pass	Pass
TC03	abc@gmail.com	123456	Login Failed	Pass	Pass
TC04	(empty)	(empty)	Login Failed	Pass	Pass
TC05	Test@gmail.com	123456	Login Failed (case sensitive)	Pass	Pass

Short Comparison

Python vs React.js + Node.js

Parameter	Python	React.js + Node.js
Type	Console application	Web application
Setup	1 file, no installation	Two projects, npm packages
Lines of code (approx.)	25	75
User interface	Text only	Browser form
Multi-user / network	No	Yes (REST API)
Best for	Scripts, learning, prototypes	Real web applications

Tools

Comparison	Key Difference
Git vs SVN	Git is distributed (works offline, fast branching); SVN is centralized.
VS Code vs IntelliJ	VS Code is a light editor for many languages; IntelliJ is a heavy full IDE, best for Java.
Jira vs Trello	Jira is for large Scrum teams; Trello is simple Kanban for small teams.
Jenkins vs GitHub Actions	Jenkins is self-hosted; GitHub Actions is cloud-based and set up using a YAML file.
Postman vs Selenium	Postman tests APIs; Selenium tests the browser UI.

Advantages and Limitations

- **Python:** simple, quick, no setup. Limitation: console only, not accessible over a network.
- **React.js + Node.js:** real web interface, scalable, same language for frontend and backend. Limitation: more setup and learning effort.
- **Static credentials:** good for learning, but not secure. Real projects use a database and hashed passwords.

Result / Conclusion

The login flow with static credentials (test@gmail.com / 123456) was successfully implemented using Python and using React.js with Node.js. Both gave the same results: "Login Successful" for correct credentials and "Login Failed" for all others. Python is better for quick and simple programs, while React.js + Node.js is better for real web applications.